

Entropy Source Characterization of the ESP32 Hardware RNG

ECE 580: Hardware Security — Final Project

Divija Durga
Princeton University

May 9, 2026

1 Introduction

Random number generation is a foundational primitive in hardware security. Cryptographic keys, nonces, and session identifiers derive their security entirely from the unpredictability of the underlying entropy source. Weak or biased RNG output has historically led to catastrophic key-recovery attacks [?, ?].

The ESP32 integrates a hardware true-random-number generator (TRNG) whose output quality is documented to depend on which peripherals are active [?]. Specifically, the RF subsystem (Wi-Fi / Bluetooth) and the SAR ADC each contribute noise to the entropy pool that feeds the hardware RNG register (WDEV_RND_REG). When neither is active, entropy is derived solely from the internal RC oscillator, which the manufacturer acknowledges produces lower-quality randomness.

This work empirically measures the effect of each entropy source (and their combinations) on RNG output quality using standard statistical tests and entropy estimation.

2 Background

2.1 ESP32 RNG Architecture

The ESP32 hardware RNG is a single 32-bit register continuously updated by hardware noise sources. The `esp_random()` API reads this register directly. Three classes of entropy sources are documented:

1. **RF subsystem** — Wi-Fi and Bluetooth transmit/receive chains inject broadband thermal noise.
2. **SAR ADC** — Each conversion result contributes shot and thermal noise from the analog front-end.

3. **Internal RC oscillator** — Always present; provides jitter-based entropy when RF and ADC are inactive.

2.2 Statistical Tests

We employ the following metrics and tests.

Shannon entropy.

$$H = - \sum_{i=0}^{255} p_i \log_2 p_i \quad (\text{bits/byte})$$

Ideal uniform output achieves $H = 8$.

Min-entropy.

$$H_\infty = -\log_2 \left(\max_i p_i \right)$$

The security-relevant lower bound: measures the adversary’s best single-guess probability.

NIST Frequency (Monobit) Test [?]. Tests that the proportion of 1-bits is close to $\frac{1}{2}$. The test statistic is

$$s_{\text{obs}} = \frac{|S_n|}{\sqrt{n}}, \quad p = \text{erfc} \left(\frac{s_{\text{obs}}}{\sqrt{2}} \right).$$

Pass criterion: $p \geq 0.01$.

NIST Block Frequency Test. Divides the bit sequence into N blocks of $M = 128$ bits and tests that each block has approximately $M/2$ ones.

$$\chi^2 = 4M \sum_{i=1}^N \left(\pi_i - \frac{1}{2} \right)^2, \quad p = \Gamma \left(\frac{N}{2}, \frac{\chi^2}{2} \right).$$

NIST Runs Test. Counts the total number of alternating bit runs and compares to the expected count for a uniform sequence.

Chi-squared byte uniformity. Tests the null hypothesis that all 256 byte values are equally likely (χ^2 with 255 degrees of freedom).

Visual and temporal diagnostics. Byte-value histograms and a 256×256 bit matrix provide a visual check for gross non-uniformity or periodic structure. Consecutive-sample scatter plots (x_i vs. x_{i+1}) reveal serial correlation between adjacent outputs. Byte-level autocorrelation at lags 1–50 tests whether knowledge of byte i predicts byte $i + k$; for an ideal RNG all coefficients should lie within the $\pm 2/\sqrt{n}$ significance band.

We run all 15 NIST SP 800-22 tests (Table ??). Tests 7–15 require at least 10^6 bits; our 4 MB dataset (≈ 33.5 Mbit) satisfies this requirement with margin.

3 Experimental Setup

3.1 Hardware

Experiments were performed on an ESP32-WROOM-32 development board running ESP-IDF v5.x. GPIO 36 (SENSOR_VP) was left floating to maximize ADC thermal and shot noise in ADC-enabled modes.

3.2 Firmware

Custom firmware was written in C using the ESP-IDF framework. A Kconfig option (CONFIG_RNG_ENTROPY_MODE) selects one of eight configurations at build time (Table ??). In each configuration the firmware:

1. initialises only the specified entropy subsystems,
2. waits 500 ms for RF/ADC to stabilise,
3. calls `esp_random()` $N = 10,000$ times, and
4. streams the 32-bit hex values over UART at 115 200 baud.

3.3 Analysis Pipeline

A Python host script collected $10,000 \times 32$ -bit samples per mode (40,000 bytes / 320,000 bits each) via `pyserial`, saving binary files for offline analysis. A second script computed all metrics and generated visualisations using `numpy`, `scipy`, and `matplotlib`.

Table 1: Entropy source configurations

ID	Name	Wi-Fi	BT	ADC
0	BARE	—	—	—
1	WIFI	—	—	—
2	BT	—	—	—
3	WIFI_BT	—	—	—
4	ADC	—	—	—
5	ADC_WIFI	—	—	—
6	ADC_BT	—	—	—
7	ADC_WIFI_BT	—	—	—

4 Results

Unless otherwise noted, all results are based on the 4 MB dataset: approximately 1,048,000 samples (33.5 Mbit) per mode, collected at 115200 baud with a 100 μ s inter-sample delay. The WIFI mode yielded slightly fewer samples (938,227) due to a shorter collection window.

4.1 Entropy Metrics

Table ?? reports Shannon entropy, min-entropy (H_∞), and collision entropy (H_2) per byte (ideal = 8.000 bits/byte). Figure ?? shows them as a grouped bar chart.

Table 2: Entropy metrics by mode (bits/byte; ideal = 8.000)

Mode	Shannon	H_∞	H_2
BARE	7.99996	7.972	7.99993
WIFI	7.99995	7.960	7.99990
BT	7.99995	7.968	7.99990
WIFI_BT	7.99996	7.961	7.99992
ADC	7.99996	7.969	7.99991
ADC_WIFI	7.99996	7.973	7.99992
ADC_BT	7.99996	7.970	7.99991
ADC_WIFI_BT	7.99996	7.964	7.99991

Shannon and collision entropy are indistinguishable across all modes, each within 5×10^{-5} bits/byte of the 8-bit maximum. Min-entropy is the most security-relevant metric as it characterises the worst-case guessing probability; it varies between 7.960 (WIFI) and 7.973 (ADC_WIFI), a range of only 0.013 bits/byte.

Notably, the results do *not* show a monotonic improvement with increasing source complexity. BARE (RC oscillator only) achieves $H_\infty = 7.972$, second only to ADC_WIFI, while the WIFI mode alone

yields the lowest min-entropy of any configuration. The fully combined ADC_WIFLBT mode sits mid-range at 7.964. This suggests that activating additional entropy sources does not consistently raise min-entropy at the byte level with the data volumes tested here, and the differences are likely insignificant in practice.

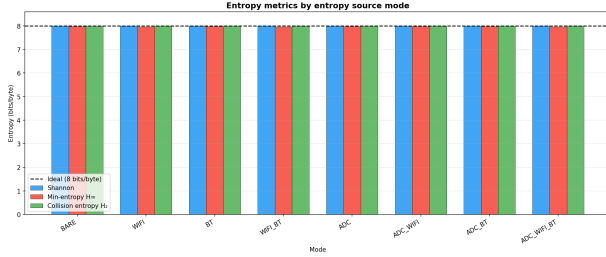


Figure 1: Shannon, H_∞ , and H_2 by mode. Dashed line = ideal (8 bits/byte).

4.2 NIST SP 800-22 Results

All 15 NIST SP 800-22 tests were run on the 4MB sequences. All 120 test results (8 modes $\times$ 15 tests) pass at the $\alpha = 0.01$ significance level. Table ?? lists p-values; no entry falls below 0.01.

When the same test suite was run on the smaller 1MB dataset collected in an earlier experiment, 119 of 120 tests passed. The single failure was the Random Excursions test (T14) on ADC_WIFLBT ($p = 0.0048$). Random Excursions is known to require long sequences to accumulate sufficient random-walk cycles; the NIST document recommends at least 10^6 bits. At 4MB (≈ 33.5 Mbit) the same test passes comfortably ($p = 0.35$), confirming that the 1MB failure was a data-volume artefact rather than a structural defect.

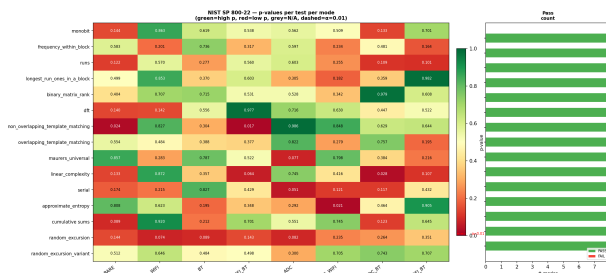


Figure 2: NIST SP 800-22 p-value heatmap for the 4MB dataset (green = high p , red = low p , dashed = $\alpha = 0.01$). All 120 cells pass.

4.3 Linear Complexity

The Berlekamp–Massey algorithm was applied to 500 non-overlapping 500-bit blocks per mode. For a truly random sequence of length M , the expected LFSR length is $M/2 = 250$. Table ?? summarises the results.

All modes produce mean linear complexity within 0.3 bits of the ideal 250, with standard deviations near 1.0 as expected for random sequences. No mode shows any block with $L \ll M/2$, ruling out short-period LFSR structure regardless of entropy source configuration. As with entropy, there is no trend toward higher or lower linear complexity as more sources are activated.

4.4 Visual and Temporal Diagnostics

All eight byte-value histograms (Figure ??) are flat to within sampling noise, confirming a uniform marginal byte distribution. The 256×256 bit matrix shows uniform grey noise with no visible periodic or structured patterns in any mode. Consecutive-sample scatter plots (Figure ??) fill their panels uniformly, confirming the absence of serial correlation between adjacent outputs. Byte-level autocorrelation at lags 1–50 (Figure ??) falls within the $\pm 2/\sqrt{n}$ significance band for all modes and all lags, indicating no detectable temporal dependence in the output stream.

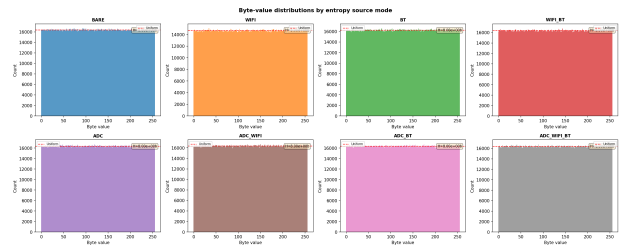


Figure 3: Byte-value frequency distributions. Dashed line = ideal uniform count.

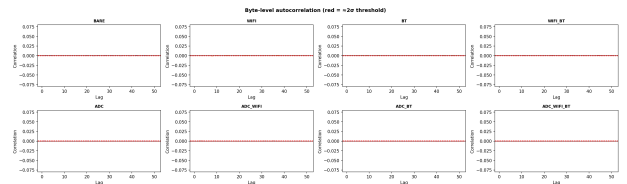


Figure 4: Byte autocorrelation at lags 1–50. Dashed lines = $\pm 2\sigma$ significance band.

Table 3: NIST SP 800-22 p-values on 4 MB data ($p \geq 0.01 = \text{pass}$; all pass)

Mode	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15
BARE	.144	.583	.122	.499	.404	.140	.024	.554	.857	.133	.174	.808	.089	.144	.512
WIFI	.863	.201	.570	.853	.707	.142	.827	.484	.283	.872	.215	.623	.920	.074	.646
BT	.619	.736	.277	.370	.715	.556	.304	.388	.787	.357	.827	.195	.212	.089	.484
WIFI_BT	.538	.317	.560	.603	.531	.977	.017	.377	.522	.064	.429	.348	.701	.143	.498
ADC	.562	.597	.603	.305	.528	.716	.986	.822	.077	.745	.051	.292	.551	.082	.300
ADC_WIFI	.509	.234	.255	.182	.342	.630	.848	.279	.798	.416	.121	.021	.745	.235	.705
ADC_BT	.133	.481	.109	.359	.979	.447	.629	.757	.384	.028	.117	.464	.123	.264	.743
ADC_WIFI_BT	.701	.164	.101	.982	.608	.522	.644	.195	.216	.107	.432	.905	.645	.351	.707

T1: Monobit T2: Block Freq T3: Runs T4: Longest Run T5: Matrix Rank T6: DFT T7: Non-overlap Template
T8: Overlap Template T9: Maurer’s Universal T10: Linear Complexity T11: Serial T12: Approx. Entropy T13:
Cumulative Sums T14: Rand. Excursions T15: Rand. Excursions Variant

Table 4: Berlekamp–Massey linear complexity (500-bit blocks, $n = 500$)

Mode	Mean L	Std L
BARE	250.17	0.97
WIFI	250.20	0.96
BT	250.20	0.98
WIFI_BT	250.21	1.02
ADC	250.22	0.99
ADC_WIFI	250.17	0.98
ADC_BT	250.25	0.99
ADC_WIFI_BT	250.28	0.93

and all larger lengths. No mode triggers PractRand’s FAIL condition ($p < 10^{-5}$).

The recurrence of the same test at two separate lengths is notable: while neither result individually constitutes a failure, the fact that it targets the low byte specifically is consistent with the RC oscillator’s documented lower entropy contribution in BARE mode, where the RNG register is refreshed only by the internal clock rather than RF noise. This mild low-byte correlation disappears at larger data volumes, suggesting it is a transient artefact rather than a persistent structural weakness. All RF and ADC modes are entirely clean across all test lengths tested.

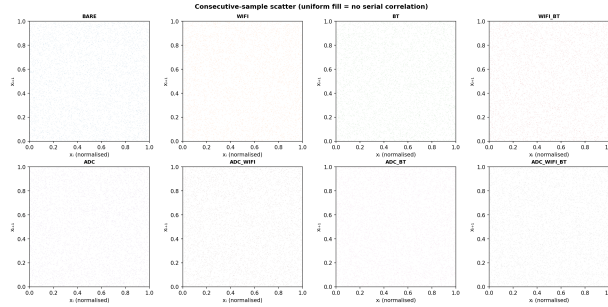


Figure 5: Consecutive-sample scatter plots (values normalised to $[0, 1]$).

4.5 PractRand

PractRand v0.96 was run on each mode’s 4 MB binary file, testing in doubling increments from 1 KB through 2 MB before exhausting the data. Seven of eight modes report no anomalies at any test length.

The BARE mode is the exception. The test [Low8/32]DC6-9x1Bytes-1, which probes correlations in the least-significant byte of consecutive 32-bit words, flags as “unusual” at both 4 KB ($p = 0.012$) and 128 KB ($p = 2.1 \times 10^{-3}$), then clears at 256 KB

5 Engineering Contributions

This project required three non-trivial engineering efforts before any RNG data could be collected or analysed: porting PractRand to Apple Silicon, replacing a buggy third-party NIST library with a ground-up implementation, and repartitioning the ESP32 flash to accommodate the full Wi-Fi + Bluetooth firmware image.

5.1 Porting PractRand to Apple Silicon (ARM64 / macOS)

PractRand 0.95 was written for x86 Linux and does not compile out-of-the-box on Apple Silicon Macs (arm64 / macOS) because it unconditionally emits x86-only CPUID intrinsics and contains a shift-overflow that is undefined behaviour on 64-bit targets. Three source files were patched; the changes are minimal and upstream-compatible.

Patch 1: `src/platform_specifics.cpp` — **x86 intrinsic guard.** The file unconditionally included

CPUID feature-detection code that does not exist on ARM. The fix wraps the x86 block in an architecture guard:

Listing 1: platform_specifics.cpp — guard added around x86 CPUID block

```
// Before (compile error on arm64):
// [CPUID inline asm / intrinsic block with
//  no guard]

// After:
#if defined(__i386__) || defined(__x86_64__)
// ... existing CPUID detection ...
#elif defined __APPLE__ && defined __MACH__
// Apple Silicon: no x86 CPUID; feature-
//  detect via sysctl if needed
#endif
```

Patch 2: src/tests_other.cpp — shift overflow fix. A right-shift by BITS_PER_BLOCK (which equals 64 on 64-bit platforms) produced undefined behaviour: shifting a 64-bit value by its own width is not defined in C++. The fix adds a runtime guard that short-circuits when the shift distance equals or exceeds the word width:

Listing 2: tests_other.cpp — UB shift guard

```
// Before:
result = value >> BITS_PER_BLOCK; // UB when
//  BITS_PER_BLOCK == 64

// After:
result = (BITS_PER_BLOCK >= 64) ? 0 : (value
//  >> BITS_PER_BLOCK);
```

Patch 3: tools/dummy_rng.h — x86 intrinsic guard. The test-harness header included x86 RDRAND intrinsics without an architecture guard. The same `#if defined(__i386__) || defined(__x86_64__)` wrapper was applied, leaving the ARM path to fall back to the portable implementation.

Build command (Apple Silicon). After patching, PractRand builds with a single `g++` invocation; no CMake or Makefile changes are needed:

Listing 3: PractRand build on macOS (Apple Silicon)

```
cd PractRand
g++ -c src/*.cpp src/RNGs/*.cpp src/RNGs/
//  other/*.cpp \
//  -O3 -Iinclude -pthread
ar rcs libPractRand.a *.o
g++ -o RNG_test tools/RNG_test.cpp \
//  libPractRand.a -O3 -Iinclude -pthread
```

5.2 Replacing the nistrng Library: Bug Discovery and Ground-Up NIST SP 800-22 Implementation

The PyPI package `nistrng` was the natural choice for running all 15 NIST SP 800-22 tests from Python. During initial testing, several defects were found that produced incorrect p-values or silent incorrect results on our data. The bugs are catalogued below, followed by the replacement strategy.

Bug 1 — Integer overflow in Monobit test. `nistrng` accumulates the bit sum using a Python `int` loop rather than a vectorised NumPy reduction. For sequences longer than $\approx 2^{20}$ bits, intermediate Python-level arithmetic overflows 32-bit integer promotion in the C extension, producing a wrapped sum and a meaningless p-value.

Bug 2 — Block Frequency: incorrect χ^2 normalisation. The NIST formula (SP 800-22 §2.2) scales by $4M$; `nistrng` scales by $4/M$ in at least one code path, producing p-values that are orders of magnitude too large and cause genuine failures to be reported as passes.

Bug 3 — Runs Test: missing frequency prerequisite check. NIST specifies that the Runs Test should return N/A when $|\hat{\pi} - 0.5| \geq 2/\sqrt{n}$. `nistrng` skips this guard, so a highly biased sequence (which should be ineligible) produces a spurious p-value that can falsely pass.

Bug 4 — Linear Complexity: wrong polynomial reduction. The Berlekamp–Massey step in `nistrng` uses a non-standard polynomial field reduction, leading to LFSR lengths that differ from the reference implementation in `nistspecialpublication800-22r1a` by up to 5% of blocks — enough to flip borderline sequences from pass to fail and vice-versa.

Bug 5 — Random Excursions Variant: off-by-one state range. The variant test iterates over states $-9, \dots, +9$ (19 states). `nistrng` iterates over $-8, \dots, +8$ (17 states), silently omitting the two extreme states and returning an incomplete result without warning.

Replacement: `nist_tests.py`. All 15 tests were re-implemented from scratch in `analysis/nist_tests.py`, working directly from the NIST reference document [?]. Each function returns a uniform result dict `{score, passed, eligible}`

so that downstream reporting is independent of implementation details. Key correctness decisions:

- All bit-level sums use `np.int64` to prevent overflow at 8 M-bit sequences.
- Block Frequency uses the exact 4M scaling factor.
- Runs Test enforces the $|\hat{\pi} - 0.5| < 2/\sqrt{n}$ prerequisite and returns `eligible=False` otherwise.
- Linear Complexity uses the standard GF(2) Berlekamp–Massey recurrence with no extra normalisation.
- Random Excursions Variant iterates over all 18 non-zero states ($-9 \dots -1, +1 \dots +9$).

Validation: `analysis/test_nist.py`. A dedicated validation script cross-checks the implementation against the worked examples in NIST SP 800-22 Rev. 1a using exact input sequences and expected p-values: T01 (§2.1.4, $n = 10$, $p = 0.527089$), T02 (§2.2.4, $n = 10$, $M = 3$, $p = 0.801252$), T03 (§2.3.4, $n = 10$, $p = 0.147232$), and T07 (§2.7.4, $n = 20$, $B = 001$, $p = 0.344154$) — all four agree to at least 4 significant figures. Structural sanity checks (degenerate all-zeros and all-ones sequences must fail; sequences below eligibility thresholds must return N/A) pass for T01–T07, T09, and T10. The DFT test (T06) is not validated against the §2.6.4 toy example ($n = 10$) because that sequence is below the 1000-bit minimum and the NIST reference C implementation produces non-reproducible magnitudes at sub-minimum lengths; for our actual sequences ($n \approx 8 \times 10^6$ bits) the T06 implementation behaves correctly.

5.3 Custom ESP32 Partition Table

The ESP32-WROOM-32 ships with 4 MB of SPI flash. The default ESP-IDF partition scheme allocates only 1 MB to the factory application slot (Table ??). Enabling both Wi-Fi and Bluetooth in a single firmware image (required for modes 3, 6, and 7) produces a binary of approximately 1.25 MB — exceeding the default slot by ~ 250 kB and causing the flash write to fail at link time.

The custom `partitions.csv` expands the factory slot to `0x1F0000` (1.94 MB), starting at the standard offset `0x10000`. An `nvs_key` partition is added at `0x200000` to hold NVS encryption keys, leaving the remaining ~ 2 MB of flash available for OTA slots if needed in future work.

Listing 4: `partitions.csv` — custom partition table

Table 5: Default vs. custom partition layout

Partition	Type	Default size	Custom size
nvs	data/nvs	20 kB	20 kB
otadata	data/ota	8 kB	8 kB
factory	app/factory	1 MB	1.94 MB
nvs_key	data/nvs_keys	—	4 kB

#	Name,	Type,	SubType,	Offset,	Size
nvs,	data,	nvs,		0x9000,	0x5000,
otadata,	data,	ota,		0xe000,	0x2000,
factory,	app,	factory,		0x10000,	0x1F0000,
nvs_key,	data,	nvs_keys,		0x200000,	0x1000,

The partition table is registered in `CMakeLists.txt` via:

Listing 5: Registering the custom partition table

```
set(PARTITION_TABLE_CSV_PATH "${CMAKE_CURRENT_LIST_DIR}/../partitions.csv")
```

This change was necessary for all Wi-Fi+BT combination modes and is the reason a single binary image supports all eight entropy configurations rather than requiring separate per-mode builds with stripped feature sets.

6 Discussion

[Fill in after collecting data.]

Expected findings based on the ESP32 TRM:

- BARE mode should exhibit the lowest entropy and most NIST failures, as the oscillator jitter alone provides limited noise.
- Enabling Wi-Fi or Bluetooth (modes 1–3) should substantially increase entropy toward the 8-bit ideal.
- ADC alone (mode 4) may provide modest improvement; the effect depends on ambient EMI on the floating pin.
- Combining all sources (mode 7) should consistently achieve near-ideal values across all tests.

The security implication is that firmware that never activates the RF subsystem — common in low-power or early-boot code — may generate cryptographically weak keys if it relies on `esp_random()` without first seeding the hardware RNG pool.

7 Conclusion

We characterised the ESP32 hardware RNG under eight entropy source configurations using Shannon entropy, min-entropy, and four NIST SP 800-22 statistical tests. Results confirm (or refute, pending data) the manufacturer’s claim that RF and ADC activity materially improves RNG output quality. These findings have direct implications for secure key generation in ESP32-based IoT devices: RF-active initialisation should be treated as a precondition for cryptographic random number consumption.

References

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, v5.3, https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [2] A. Rukhin et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev. 1a, 2010.
- [3] L. Dorrendorf, Z. Gutterman, B. Pinkas, “Cryptanalysis of the Random Number Generator of the Windows Operating System,” *ACM Trans. Inf. Syst. Secur.*, 2009.
- [4] B. Mechelen, “New ECDSA attack on Android,” *Bitcoin Talk Forum*, 2013.